

	🌐	fr	vi	zh	ru	ar	hi	avg
(Conneau et al., 2018)	⊖	67.7	66.4	65.8	65.4	64.8	64.1	65.7
(Artetxe and Schwenk, 2019)	⊕	71.9	72.0	71.4	71.5	71.4	65.5	70.6
(Lample and Conneau, 2019) (MLM)	⊖	76.5	72.1	71.9	73.1	68.5	65.7	71.3
(Lample and Conneau, 2019) (MLM+TLM)	⊕	78.7	76.1	76.5	75.3	73.1	69.6	74.9
mBERT (Wu and Dredze, 2019)	⊖	73.8	69.5	69.3	69.0	64.9	60.0	67.8
RAMEN _{BASE}								
	⊖	75.2	71.8	70.7	71.1	69.3	62.8	70.1
+ BERT	⊕	75.1	72.5	71.9	70.8	69.7	63.5	70.6
	⊖	79.9	75.9	73.7	73.6	71.9	65.6	73.4
+ RoBERTa	⊕	80.3	75.6	76.2	75.8	73.1	68.1	74.9
RAMEN _{LARGE}								
	⊖	78.1	74.8	74.5	73.7	70.8	64.5	72.7
+ BERT	⊕	78.0	75.1	71.3	74.0	71.8	66.1	72.7
	⊖	81.3	76.2	76.3	75.6	73.5	64.5	74.6
+ RoBERTa	⊕	81.0	76.2	76.8	75.0	72.9	68.2	75.0

Table 1: Zero-shot classification results on XNLI. ⊕ indicates parallel data is used. RAMEN only uses parallel data for initialization. The best results are marked in **bold**.

We take the top 50,000 words in each aligned word-vectors to compute subword vectors.

In both cases, not all the words in the foreign vocabulary can be initialized from the English word-embeddings. Those words are initialized randomly from a Gaussian $\mathcal{N}(0, 1/d^2)$.

3.3 Hyper-parameters

In all the experiments, we tune RAMEN_{BASE} for 120,000 updates and RAMEN_{LARGE} for 300,000 updates. The sequence length is set to 256. For tuning bilingual LMs, we use a mini-batch size of 112 for RAMEN_{BASE} and 24 for RAMEN_{LARGE} where half of the batch are English sequences and the other half are foreign sequences. This strategy of balancing mini-batch has been used in multilingual neural machine translation (Firat et al., 2016; Lee et al., 2017).

We optimize RAMEN using Adam optimizer. We linearly increase the learning rate from 10^{-7} to 10^{-4} in the first 4000 updates and then follow an inverse square root decay. When fine-tuning RAMEN on XNLI and UD, we use a mini-batch size of 32, Adam’s learning rate of 10^{-5} . The number of epochs are set to 4 and 50 for XNLI and UD tasks respectively.

All experiments are carried out on a single Tesla V100 16GB GPU. Each RAMEN_{BASE} model is trained within a day and each RAMEN_{LARGE} is trained within two days.¹⁰

¹⁰22 and 46 GPU hours, to be precise. Learning alignment with fast-align takes less than 2 hours and we do not account for training time of fastText vectors.

4 Results

In this section, we present the results of our models for two zero-shot cross lingual transfer tasks: XNLI and universal dependency parsing.

4.1 Cross-lingual Natural Language Inference

Table 1 shows the XNLI test accuracy. For reference, we also include the scores from the previous work, notably the state-of-the-art system XLM (Lample and Conneau, 2019). Before discussing the results, we spell out that the fairest comparison in this experiment is the comparison between mBERT and RAMEN_{BASE}+BERT trained with monolingual only. We first discuss the transfer results from BERT. Initialized from fastText vectors, RAMEN_{BASE} slightly outperforms mBERT by 2.6 points on average and widens the gap of 4.4 points on Arabic. RAMEN_{BASE} gains extra 0.3 points on average when initialized from parallel data. With triple number of parameters, RAMEN_{LARGE} offers an additional boost in term of accuracy and initialization with parallel data consistently improves the performance. It has been shown that BERT_{LARGE} significantly outperforms BERT_{BASE} on eleven English NLP tasks (Devlin et al., 2019), the strength of BERT_{LARGE} also shows up when adapted to foreign languages.

Transferring from RoBERTa leads to better zero-shot accuracies. With the same initializing condition, RAMEN_{BASE}+RoBERTa outperforms RAMEN_{BASE}+BERT on average by 3.1 and 4.3 points when initializing from monolingual and par-